Шаг 1

Чему учимся

В этом проекте вы собираете первую настоящую автоматизацию — программу, которая берёт на себя работу, до сих пор выполняемую в компаниях руками. Пройденные темы наконец сходятся в одну задачу — файлы и папки, массивы и объекты, работа со строками, аргументы командной строки, окружение и Git.

Отдельный навык, которого не даёт ни одна практика: задача приходит не в виде сигнатуры функции, а в виде папки с чужими файлами и просьбы «разберись». Решение придётся разложить на части самому.

Описание проекта

Небольшая компания годами складывала документы в общую папку. Там счета, чеки, договоры, банковские выписки, сканы и посторонние файлы, а рядом — выгрузки контактов из вебинаров, старой CRM, конференций и рассылок. Имена файлов ничего не говорят, часть данных лежит в двух или трёх копиях, форматы у выгрузок разные.

Ваша программа приводит это в порядок. У неё две команды:

```
file-automation files <папка> [--out <папка>]      # реестр документов
file-automation contacts <папка> [--out <папка>]   # чистая таблица контактов
```

Команда `files` проходит по папке, определяет тип каждого файла, находит копии, отделяет посторонние файлы от документов и собирает реестр. Команда `contacts` читает все выгрузки контактов, приводит колонки к единому виду, нормализует телефоны и адреса, убирает повторы и собирает одну чистую таблицу. В конце программа пишет отчёт, который можно отдать компании.

Вот как это выглядит в работе:



Тот же прогон текстом, чтобы разглядеть числа:

```
$ file-automation files ./company-files --out ./out

Файлов найдено: 40

Табличных документов: 21, текстовых: 8, непарсимых: 11

Документов: 28, требуют ручного разбора: 7, посторонних: 5

Копий найдено: 5 в 3 группах

Реестр: out/registry.csv

Отчёт: out/report.txt

Отчёт для программ: out/report.json
```

```
$ file-automation contacts ./company-files --out ./out

Выгрузок прочитано: 5, записей: 155

Уникальных контактов: 117

Отбраковано значений: 11, записей без телефона и почты: 5

Таблица: out/contacts.csv

Отчёт: out/report.txt
Отчёт для программ: out/report.json
```

Задачи

1. Разберитесь с идеей проекта: на входе папка с чужими файлами, на выходе реестр, чистая таблица контактов и отчёт в двух видах, для человека и для программы.
2. Посмотрите на пример вывода выше и представьте, из каких частей будет состоять программа.

Результат шага

Вы понимаете, что предстоит собрать и какими двумя командами это делается.

Шаг 2

Окружение и файлы компании

Перед кодом нужно рабочее место. Понадобятся редактор, Node.js и Git: проверьте, что `node --version` отвечает, и создайте репозиторий проекта.

Дальше — данные, на которых программа будет работать. Набор готовый, собирать его руками не нужно:

```
git clone --depth 1 https://github.com/hexlet-components/data-company-files.git
```

```
mv data-company-files/company-files company-files
rm -rf data-company-files
```

Папка набора лежит внутри репозитория, поэтому её достают и кладут рядом с кодом. Дальше в командах фигурирует именно *company-files/*.

Набор можно скачать и архивом, если так удобнее — ссылка есть на его странице. Папку *company-files/* сразу добавьте в *.gitignore*: это чужие данные, в вашем репозитории им не место.

Оформление пакета

Программа, которую вы пишете, — это пакет. Даже если её никуда не публикуют, оформляется она так же, как любой пакет Node.js, с *package.json*, точкой входа и настроенным линтером. Этим шагом вы получаете запускаемую программу, у которой ещё нет логики, зато есть всё, чтобы её добавлять.

У пакета с консольной программой есть ключ `bin`. В нём объявлено, каким именем программа запускается и какой файл при этом выполняется:

```
{
  "bin": {
    "file-automation": "bin/file-automation.js"
  }
}
```

После `npm link` это имя появляется в системе, и программа запускается как любая другая утилита:

```
npm link
file-automation --help
```

Имя выбираете вы, дальше в примерах будет `file-automation`. Файл из `bin` начинается со строки `#!/usr/bin/env node`, иначе система не знает, чем его выполнять.

Отсюда разделение, которое пригодится и дальше. В *bin/* лежит разбор командной строки и печать на экран. Логика живёт в модулях рядом, и точка входа пакета их собирает. Так вашу программу можно позвать и из другого кода, и из терминала.

Справка

Программа, запущенная без аргументов, объясняет, как ей пользоваться. Это не украшение, потому что справка это единственный способ узнать список команд, не читая код.

Писать её руками не нужно. Разбор командной строки и справку берёт на себя библиотека, например commander, и справка собирается из описаний, которые вы задали командам и параметрам:

```
$ file-automation
Usage: file-automation [options] [command]

Автоматизация разбора файлов компании

Options:
  -V, --version                output the version number
  -h, --help                   display help for command

Commands:
  files [options] <папка>      собрать реестр документов папки
  contacts [options] <папка>   собрать чистую таблицу контактов
  help [command]               display help for command
```

Библиотеку выбираете сами, и написать разбор руками тоже можно. Важно, чтобы в справке были обе команды и параметр `--out`.

Ошибки

Программу запускают неправильно чаще, чем правильно, поэтому два случая обрабатываются с самого начала. Библиотека разбора закрывает оба сама:

```
$ file-automation registry ./company-files
error: unknown command 'registry'
# дальше справка со списком команд

$ file-automation files
error: missing required argument 'папка'
```

Оба сообщения уходят в поток ошибок, минуя обычный вывод, и программа завершается ненулевым кодом. Так вызывающая сторона, будь то CI, скрипт или другая программа, узнаёт, что запуск не удался.

Задачи

1. Подготовьте окружение: редактор, Node.js, Git. Создайте репозиторий проекта и склонируйте его себе.
2. Получите папку с файлами компании и положите её рядом с кодом. Добавьте *company-files/* в *.gitignore*.
3. Создайте *package.json*. Укажите тип модулей, чтобы работали `import` и `export`, и объявите ключ `bin` с именем программы.
4. Создайте файл из `bin` с шебангом. Заведите в нём две команды, `files` и `contacts`, каждая принимает папку и параметр `--out` со значением по умолчанию *./out*.
5. Выполните `npm link` и убедитесь, что программа запускается по имени и печатает справку.
6. Проверьте, что неизвестная команда и запуск команды без папки дают сообщение в поток ошибок и ненулевой код выхода.
7. Заведите заготовки модулей под обе команды — пока они только сообщают, с какой папкой их вызвали.
8. Настройте линтер и добейтесь, чтобы он проходил без ошибок. Инструмент выбираете сами.
9. Опишите в *README.md*, как программу установить и запустить.

Требования

- Папка с файлами компании лежит рядом с кодом и не попадает в Git.
- Запуск без аргументов печатает справку и завершается кодом `0`.
- Неизвестная команда и команда без папки печатают сообщение в поток ошибок и завершаются ненулевым кодом.
- Значение `--out` по умолчанию — *./out*.
- Линтер запускается одной командой и проходит без ошибок.

Подсказки

- Аргументы запуска лежат в `process.argv`, первые два элемента — путь к Node.js и путь к скрипту.
- Поток ошибок — это `console.error`, код выхода задаёт `process.exit`.
- Проверить, что ключ есть в объекте, помогает `Object.hasOwn`.

Результат шага

Окружение готово, данные компании лежат рядом с кодом, `file-automation` печатает справку, неверный запуск завершается ошибкой, линтер зелёный.

Шаг 3

Результат, который можно показать

Реестр и чистая таблица — это файлы для машины. Их открывают в редакторе таблиц, загружают в другую систему, отдают бухгалтеру. А компании, которая отдала вам папку, нужен ответ на человеческом языке. Что у вас в этой папке лежит, сколько там мусора и сколько работы осталось людям.

Последний шаг собирает отчёт, и собирает его в двух видах. Один, *report.txt*, читает человек. Второй, *report.json*, читает программа.

Отчёт для человека

report.txt открывается прямо в браузере или в редакторе и читается за минуту. Он отвечает на вопросы, которые задаст любой, кто получит результат вашей работы.

Числа тут мало что значат сами по себе. «Требуют ручного разбора: 7» не говорит, чей это рабочий день и с чего начать, поэтому отчёт для человека **называет файлы**:

```
Требуют ручного разбора (7)

Программа эти файлы не прочитала, содержимое придётся посмотреть человеку.

  akt-priyomki.pdf
  skan-dogovora.jpg
  Архив/akt-2024.pdf
```

Имена пишутся с путём от папки компании. Одинаково названные файлы лежат в разных подпапках, и по одному имени человек не поймёт, какой из них открывать.

Три вещи, ради которых отчёт и читают:

- какие файлы требует ручного разбора, потому что это чей-то рабочий день и его надо запланировать;
- какие файлы можно удалить, потому что они посторонние;
- какие файлы лежат копиями и сколько места освободится, потому что это второй аргумент за уборку.

Какую копию из группы оставить, отчёт не советует. Правило выбора основной копии вы придумали сами на шаге про дубликаты, и оно произвольное, поэтому решает человек. Отчёт показывает состав группы и лишний размер.

Плюс к этому контакты: сколько получилось, из каких источников и сколько записей отбраковано.

Выровненные колонки

Сводку по источникам читать удобнее колонками:

```
Источник                 Записей
------------------------------
anketa-konferencii.csv   32
crm-arhiv.json           27
kontakty-crm.csv         36
kontakty-vebinar.csv     36
kontakty-rassylka.csv    24
```

Выравнивать руками не нужно. Считать длину самого длинного значения, добивать остальные пробелами, рисовать линейку под заголовком. Работа известная, и пакетов под неё написано много. В эталоне взята as-table, тот же пакет используют курсы Хекслета по SQL.

```
npm install as-table
```

Она принимает массив объектов и возвращает **строку**. Заголовок берётся из ключей, ширина колонок считается по содержимому:

```
import asTable from "as-table";

const files = [
  { Файл: "dogovor-arendy.txt", Размер: 4096 },
  { Файл: "svodnaya-tablica.xlsx", Размер: 18944 },
];

console.log(asTable(files));
// Файл                  Размер
// ---------------------------
// dogovor-arendy.txt     4096
// svodnaya-tablica.xlsx  18944
```

Массив массивов она тоже принимает, и тогда заголовка и линейки не будет. Так удобно печатать данные без подписей.

Раз она возвращает строку, вставляется результат туда же, куда и остальные куски отчёта, в общий массив строк перед `join`.

Пакет выбираете вы, таких много. Написать выравнивание самому тоже можно, и на латинице с кириллицей хватит `padEnd` по длине строки. Ломается такой способ на эмодзи и на цветном выводе, потому что там длина строки перестаёт совпадать с числом занятых в терминале клеток, и колонки разъезжаются. Готовые пакеты этот случай уже учли.

Списки файлов в таблицу не загоняйте. Файлы под ручной разбор и группы копий — это списки, у них одна колонка, и таблица им ничего не добавит. Библиотека нужна там, где колонок правда несколько.

Отчёт для программы

report.json нужен затем же, зачем любой структурированный вывод. Его читает следующая программа, та, что загрузит подготовленные данные дальше, построит график или отправит задачу в трекер. Разбирать ради этого сплошной текст никто не станет.

Структуру придумываете вы, и здесь она не приводится. Требование одно: это должен быть разбираемый непустой JSON. Что в нём лежит и как называются ключи, решаете вы.

Отчёт один на обе команды

Отчёт описывает папку целиком, а команды у вас две, и каждая знает только свою половину. Поэтому вторую половину отчёт добирает сам, тем же кодом, которым её считает вторая команда.

Так отчёт получается полным независимо от того, какую команду запустили последней. Отсюда же требование шага отделить подсчёт от печати. Числа нужны трижды, терминалу и двум отчётам, и посчитаны они должны быть один раз.

```
$ file-automation files ./company-files --out ./out
Файлов найдено: 40
Табличных документов: 21, текстовых: 8, непарсимых: 11
Документов: 28, требуют ручного разбора: 7, посторонних: 5
Копий найдено: 5 в 3 группах
Реестр: out/registry.csv
Отчёт: out/report.txt
Отчёт для программ: out/report.json
```

Задачи

1. Отделите подсчёт от печати, если ещё не отделили. Одни и те же числа нужны терминалу и обоим отчётам.
2. Соберите *report.txt* из разделов: документы, копии, контакты. Файлы в нём назовите по именам с путём.
3. Сводку по источникам выровняйте колонками. Возьмите готовый пакет, не пишите подсчёт ширин сами.
4. Соберите *report.json* из тех же данных. Структуру выбираете сами.
5. Доберите вторую половину чисел тем же кодом, которым её считает вторая команда.
6. Запишите оба отчёта в папку из `--out` и сообщите в терминале пути к ним.

Требования

- *report.txt* и *report.json* появляются в папке из `--out` при запуске любой из двух команд.
- В *report.txt* перечислены файлы под ручной разбор и файлы на удаление.
- В *report.txt* показаны группы копий и занимаемое ими место.

- В *report.txt* есть числа по контактам и перечислены источники.
- *report.json* разбирается как JSON и не пустой.
- Числа в отчётах совпадают с тем, что программа напечатала в терминале.
- Файлы из вложенных папок входят в оба отчёта.

Подсказки

- Массив строк и `join("\n")` — достаточный способ собрать документ, шаблонизатор не нужен.
- Для json готовое есть в стандартной библиотеке, это `JSON.stringify`. Второй и третий аргументы включают отступы, и файл становится читаемым глазами.
- Путь от папки компании даёт `path.relative`.
- Размер копий — это сумма размеров файлов группы без основной копии.

Результат шага

`file-automation files ./company-files --out ./out` и `file-automation contacts ./company-files --out ./out` создают *out/report.txt* и *out/report.json*. В первом видно, что лежит в папке компании и что с этим делать, второй готов для следующей программы. Проект готов.

Шаг 4

Список файлов

Разбор начинается с простого вопроса: что вообще лежит в папке компании. Программа проходит по ней и собирает по каждому файлу четыре вещи — путь, имя, расширение и размер. Дальше из этих данных вырастает весь реестр: тип определяется по расширению и содержимому, копии ищутся по имени и размеру, в отчёт идут числа.

Пока задача ровно одна — получить список и напечатать, сколько файлов нашлось:

```
$ file-automation files ./company-files
Файлов найдено: 34
```

Одно замечание на будущее. Пусть подсчёт числа возвращает, а печатает вызывающая сторона. В конце проекта эти же числа уйдут в оба отчёта, и считать их заново не придётся.

Вложенные папки пока пропускаем

В папке компании есть подпапка *Архив/*, и её содержимое в список не попадает — намеренно. Обход станет рекурсивным в конце проекта, отдельным шагом: так вы сначала получаете работающий результат на плоском списке, а потом видите, во что обходится включение вложенности, когда команд уже две.

Из этого следует практический совет: держите обход папки в одной функции и не растаскивайте его по коду. Позже вы поменяете эту функцию один раз, и вложенность подхватят обе команды.

Задачи

1. Соберите список файлов папки, которую передали команде.
2. Сохраните по каждому файлу путь, имя, расширение и размер.
3. Пропустите вложенные папки: в список идут только файлы верхнего уровня.
4. Напечатайте, сколько файлов нашлось.

Требования

- Команда `files` печатает число файлов папки.
- В список попадают только файлы: сами папки в нём не учитываются.
- Файлы из вложенных папок пока не считаются.
- У каждого файла собраны путь, имя, расширение и размер.

Подсказки

- `readdirSync` с параметром `withFileTypes` возвращает записи, у которых есть `isFile()` и `isDirectory()`.
- Размер файла лежит в `statSync`, полный путь собирает `path.join`.

- Расширение достаёт `path.extname`. У файла без расширения оно пустое, и у *.DS_Store* тоже: имя, начинающееся с точки, расширением не считается.

Результат шага

`file-automation files ./company-files` печатает число файлов папки компании, файлы из *Архив/* в него не входят.

Шаг 5

Имена файлов врут

Папка собиралась годами и разными людьми, поэтому доверять именам нельзя: скан лежит под именем документа, у справки расширения нет вовсе, а *.DS_Store* вообще не файл компании. Поэтому тип определяется по двум признакам сразу — по расширению и по первым байтам файла.

Первые байты — это подпись формата. Она стоит в начале файла независимо от того, как файл назвали:

Формат	Первые байты
pdf	`%PDF`
jpeg	`ÿØÿ`
xlsx	`PK` — это zip-архив внутри

Категорий три:

- **табличный документ** — csv и json: из них на следующих шагах берутся строки и колонки;
- **текстовый документ** — txt и md: из них в реестр идут только метаданные;
- **непарсимый** — pdf, сканы, xlsx: их содержимое программа не читает.

Тип получают все файлы папки без исключения. Файл, оставшийся без типа, до реестра не доедет, и в отчёте его не будет — а компании нужен ответ по каждому файлу, в том числе «этот трогать не стали».

Постороннее и ручной разбор — разные вещи

Кроме типа у файла есть статус: что с ним делать дальше. Тут легко всё свалить в кучу «не смогли прочитать», и делать этого нельзя, потому что за двумя случаями стоят разные действия.

Постороннее — файл, который документом не является вовсе: системный мусор вроде *.DS_Store*, пустой файл, расширение вне списка тех, с которыми программа работает. Компании про такой файл нужно знать одно — что он есть и что его пропустили.

Требует ручного разбора — файл, который документ, но программой не читается: акт сверки в pdf, скан чека, выгрузка в xlsx. Это работа для человека, и по таким файлам компания планирует чей-то день.

Всё остальное — обычный **документ**, с которым программа работает дальше сама.

Скан, попавший в папку, программу не роняет: он получает свой статус и остаётся в разборе наравне с остальными.

```
$ file-automation files ./company-files
Файлов найдено: 34
Табличных документов: 18, текстовых: 6, непарсимых: 10
Документов: 23, требуют ручного разбора: 6, посторонних: 5
```

Задачи

1. Определите тип каждого файла из списка: табличный документ, текстовый документ или непарсимый.
2. Используйте оба признака — расширение и первые байты. Когда они спорят, верить нужно байтам.
3. Определите тип файла без расширения: расширения нет, значит остаются только байты.
4. Расставьте статусы: постороннее, требует ручного разбора, документ.

5. Дополните сводку в выводе: сколько файлов каждой категории и каждого статуса.

Требования

- У каждого файла папки есть тип и статус — файлов без них не остаётся.
- Тип определяется по расширению и по первым байтам, подпись формата важнее имени.
- Файл без расширения получает тип по первым байтам.
- Системный мусор и пустые файлы получают статус «постороннее».
- Файлы, которые программа не читает, получают статус «требует ручного разбора» и не смешиваются с посторонними.
- Скан в папке не прерывает работу программы.
- В выводе есть сводка по категориям и по статусам, числа в ней сходятся с числом файлов.

Подсказки

- Первые байты даёт `readFileSync`: у полученного значения есть `subarray`, чтобы взять начало.
- Байт сравнивается с числом, `%` — это `0x25`, `P` — `0x50`. Проверить начало файла помогает `every` по массиву байтов.
- Расширения и подписи форматов лучше собрать в одном месте — объектом или массивом, а не строками внутри условий: список пополняется чаще, чем логика разбора.
- Проверить, есть ли ключ в объекте, помогает `Object.hasOwn`.

Результат шага

`file-automation files ./company-files` печатает сводку: сколько файлов каждой категории и сколько документов, файлов на ручной разбор и посторонних. Числа сходятся с общим числом файлов.

Шаг 6

Одно и то же под разными именами

Папка собиралась годами, файлы в неё складывали разные люди, поэтому один документ лежит в ней по два и по три раза. Компании важно знать не только, что копии есть, но и какой файл из группы оставить.

Копии бывают двух родов, и ищутся они по-разному.

Побайтовая копия — файл, содержимое которого совпадает с другим до последнего байта, а имя может быть любым. Сравнивать содержимое напрямую дорого и неудобно, поэтому от каждого файла считают хеш — короткую строку, которая меняется при изменении даже одного байта. Совпали хеши — совпало содержимое.

Копия по имени — файл, который назвали почти так же. Содержимое при этом отличается: кто-то дописал строку и сохранил рядом. Такие имена узнаются после приведения к одному виду:

Имя в папке	После приведения
СЧЁТ.csv	*счём.csv*
Счёт (1).csv	*счём.csv*
Копия счём.csv	*счём.csv*

Основная копия

Копии собираются в группы, и в каждой группе один файл объявляется основной копией — тот, который компания оставит. Остальные файлы группы на неё ссылаются: в реестре у них будет путь основной копии, у самой основной — пусто.

Правило выбора основной копии придумываете вы, но оно должно давать один и тот же ответ при каждом запуске. Порядок, в котором файлы приходят из файловой системы, такой гарантии не даёт: отсортируйте список, прежде чем выбирать.

```
$ file-automation files ./company-files
Файлов найдено: 34
Табличных документов: 18, текстовых: 6, непарсимых: 10
Документов: 23, требуют ручного разбора: 6, посторонних: 5
```

`Копий найдено: 4 в 2 группах`

Задачи

1. Посчитайте хеш содержимого каждого файла.
2. Найдите файлы с одинаковым хешем — это побайтовые копии.
3. Приведите имена к одному виду: нижний регистр, без суффикса `(1)`, без слова «копия».
4. Найдите файлы, у которых совпало приведённое имя.
5. Соберите копии в группы, выберите в каждой основную и запомните у остальных ссылку на неё.
6. Добавьте в сводку число копий и число групп.

Требования

- Хеш считается по содержимому файла, а не по имени и размеру.
- Файлы с одинаковым содержимым попадают в одну группу, даже если названы по-разному.
- Файлы, чьи имена различаются только регистром, суффиксом `(1)` или словом «копия», попадают в одну группу.
- В группе ровно одна основная копия, остальные файлы ссылаются на неё.
- Ссылки не выстраиваются в цепочку: копия ссылается на основную, а не на другую копию.
- Выбор основной копии не зависит от порядка, в котором файлы пришли из файловой системы.
- В выводе есть число найденных копий и число групп.

Подсказки

- Хеш даёт `createHash` из модуля `node:crypto`: `createHash("sha256").update(содержимое).digest("hex")`.
- Расширение при сравнении имён лучше не терять: *счёт.csv* и *счёт.txt* — разные документы.
- `Map` удобно хранит «уже виденный хеш → путь основной копии»: один проход по отсортированному списку, и группы собраны.

- Отсортировать список файлов помогает `sort`, сравнивать пути можно как строки.

Результат шага

`file-automation files ./company-files` сообщает, сколько копий нашлось и в скольких группах, и повторный запуск даёт тот же ответ.

Шаг 7

Реестр

Всё, что программа узнала о файлах, пора выгрузить в таблицу. Реестр — это csv-файл, по строке на каждый файл папки. Его открывают в редакторе таблиц, по нему сортируют и фильтруют, его отдают бухгалтеру. После этого шага первая команда работает целиком.

Колонки реестра такие:

Колонка	Что в ней
путь	путь к файлу
имя	имя файла
тип	табличный документ, текстовый документ или непарсимый
размер	размер в байтах
хеш	хеш содержимого
статус	документ, требует ручного разбора или постороннее
основная копия	путь основной копии, у самой основной пусто
строк	число строк табличного документа

колонки	названия колонок табличного документа

Строки в реестр попадают все, включая посторонние файлы и те, что требуют ручного разбора: компании нужен ответ по каждому файлу, в том числе «этот трогать не стали».

Полезное — из содержимого

Две последние колонки заполняются только у табличных документов, и заполняются они содержимым, а не названием. Программа открывает csv или json, берёт заголовок и считает строки:

- у csv первая строка — названия колонок, остальные строки данных;
- у json это ключи объектов, а строки — элементы массива.

Тут вылезают следы Excel. Часть выгрузок начинается с невидимого служебного символа BOM, а разделителем в них стоит точка с запятой вместо запятой. Заголовок, прочитанный без учёта этого, превращается в одну колонку с мусором в начале.

Куда писать

Результат уходит в папку из параметра `--out`, по умолчанию — *./out*. В папку компании программа не пишет ничего: это чужие данные, и мусорить в них нельзя.

```
$ file-automation files ./company-files --out ./out
Файлов найдено: 34
Табличных документов: 18, текстовых: 6, непарсимых: 10
Документов: 23, требуют ручного разбора: 6, посторонних: 5
Копий найдено: 4 в 2 группах
Реестр: out/registry.csv
```

Задачи

1. Прочитайте у табличных документов заголовок и посчитайте строки.
2. Учтите BOM в начале файла и точку с запятой вместо запятой.
3. Соберите строку реестра по каждому файлу папки.

4. Запишите *registry.csv* в папку из `--out`, создав её, если её нет.
5. Допишите в сводку путь к готовому реестру.

Требования

- *registry.csv* появляется в папке из `--out`, по умолчанию — в *./out*.
- Папка результата создаётся программой, если её не было.
- В реестре первая строка — названия колонок, дальше по строке на каждый файл папки.
- Колонки идут в порядке из таблицы выше.
- У табличных документов заполнены число строк и названия колонок, у остальных эти колонки пустые.
- Значение с запятой внутри не разъезжается по соседним колонкам.
- Программа не создаёт файлов в папке компании.

Подсказки

- Папку создаёт `mkdirSync` с параметром `recursive`, файл пишет `writeFileSync`.
- BOM — это символ в самом начале строки; его достаточно отрезать.
- Разделитель можно определить по заголовку: сравните, чего в нём больше — запятых или точек с запятой.
- Значение, внутри которого есть запятая, в csv заключают в двойные кавычки.

Результат шага

`file-automation files ./company-files --out ./out` создаёт *out/registry.csv*, и таблица открывается в редакторе: по строке на файл, у табличных документов видны строки и колонки.

Шаг 8

Выгрузки контактов

Вторая команда занимается контактами. Компания годами собирала их из разных мест: вебинары, старая CRM, конференции, рассылки. Каждый источник выгружал

свой файл, и колонки в этих файлах называются по-разному — *Телефон*, *Моб. телефон*, *phone*; *E-mail*, *Почта*, *mail*.

Задача шага — прочитать все выгрузки папки и привести их колонки к одной схеме: имя, телефон, почта, источник. Нормализацией значений займётся следующий шаг, здесь важно собрать записи вместе.

Какие файлы читать

Перечислять имена выгрузок в коде нельзя: в папке компании их четыре, а завтра появится пятая, и имя у неё будет другое. Искать по слову «контакты» в имени тоже не выйдет — имена в этой папке врут, вы это уже видели.

Надёжный признак один — колонки. Табличный документ, среди колонок которого есть телефон или почта, и есть выгрузка контактов. Счёт и банковская выписка тоже табличные документы, но ни телефона, ни почты в них нет, и в контакты они не попадут.

Карта соответствий

Названия колонок сводит к схеме карта соответствий — обычный объект, где у каждого поля схемы перечислены известные названия:

```
const columnMap = {
  имя: ["имя", "фио", "name"],
  телефон: ["телефон", "моб. телефон", "phone"],
  почта: ["почта", "e-mail", "email"],
};
```

Карта лежит отдельно от логики разбора, и это главное в шаге. Пятый источник с шестым названием колонки добавляется дописыванием строки в карту, а не правкой функций.

Колонки, которых в карте нет, не выбрасываются молча — программа их называет. Молча потерянная колонка — это молча потерянные данные: никто не заметит, что из выгрузки не приехали должности или города.

```
$ file-automation contacts ./company-files
anketa-konferencii.csv: записей 32
```

```
crm-arhiv.json: записей 27
kontakty-crm.csv: записей 36
kontakty-vebinar.csv: записей 36
Выгрузок прочитано: 4, записей: 131
Колонки без соответствия: Должность (anketa-konferencii.csv), company (crm-arhiv.json)
```

Задачи

1. Соберите табличные документы папки — теми же средствами, что и в первой команде.
2. Отберите из них выгрузки контактов: те, где среди колонок есть телефон или почта.
3. Заведите карту соответствий: поле схемы — известные названия колонок.
4. Прочитайте записи каждой выгрузки, разложив значения по полям схемы.
5. Запомните у каждой записи источник — имя файла, из которого она приехала.
6. Напечатайте, сколько записей прочитано из каждой выгрузки и сколько всего.
7. Назовите колонки, которым не нашлось соответствия в карте.

Требования

- Выгрузки определяются по колонкам, имена файлов в коде не перечислены.
- Табличный документ без телефона и почты выгрузкой контактов не считается.
- Карта соответствий лежит отдельно от логики разбора: добавление названия колонки не требует править функции.
- Разные названия одной и той же колонки приводятся к одному полю схемы.
- У каждой записи известен источник.
- Колонки без соответствия названы в выводе.
- BOM в начале файла и точка с запятой вместо запятой чтение не ломают.

Подсказки

- Чтение табличного файла у вас уже написано в первой команде — вынесите его в общий модуль и зовите из обеих команд.

- Сравнивать названия колонок лучше в нижнем регистре и без лишних пробелов: в выгрузках встречается и *Телефон*, и то же слово с пробелом на конце.
- `Object.keys` по карте даёт список полей схемы, `includes` проверяет, знакомо ли название колонки.
- У записи из json колонки — это ключи объекта.

Результат шага

`file-automation contacts ./company-files` печатает, сколько записей прочитано из каждой выгрузки, сколько всего, и какие колонки остались без соответствия

Шаг 9

Одно и то же разными словами

Записи собраны, но сравнивать их пока нельзя. Один и тот же телефон в четырёх выгрузках записан четырьмя способами, почта приехала с лишними пробелами и заглавными буквами, а вместо пустого значения кое-где стоит прочерк.

Как записано	Что это значит
`8-921-555-14-95`, `+7 (921) 555-14-95`, `89215551495`	один и тот же номер
`USER001@EXAMPLE.COM`, `user001@example.com`	один и тот же адрес
`АЛИСА НЕСТЕРОВА`, `Алиса Нестерова`	одно и то же имя
`-`, `NULL`, пустая ячейка	значения нет

Нормализация приводит значение к одному виду, чтобы такие записи можно было сравнивать. Без неё склейка на следующем шаге найдёт четверых разных людей там, где человек один.

Телефон

Телефон — это цифры. Всё остальное — скобки, дефисы, пробелы, плюс — оформление, и его достаточно отфильтровать. Дальше остаётся привести начало номера к одному виду: российский номер записывают и с `8`, и с `+7`, и вовсе без кода страны.

Регулярные выражения тут не нужны, и это не упрощение: фильтр цифр — это `filter` по строке, то есть ровно то, что вы делали с массивами весь модуль.

Номер, в котором цифр не хватает, нормализацию не проходит. Такое значение отбраковывается: в таблицу оно не попадает, но программа о нём сообщает, потому что молча потерянные данные хуже потерянных заметно.

Почта и имя

Почта приводится к нижнему регистру и обрезается по краям. Адрес без `@` адресом не является и отбраковывается так же, как короткий телефон.

Имя чистится от двойных пробелов и приводится к обычному виду: первая буква слова заглавная, остальные строчные. Так `иван петров` и `ИВАН ПЕТРОВ` перестают быть разными строками.

Пустое остаётся пустым

Отдельное правило, которое кажется мелочью, пока таблицу не начнут загружать в другую систему: пустое значение остаётся пустым. Не `-`, не `нет данных`, не `null` и не `NULL` — пустая ячейка.

Прочерк вместо пустоты — это отсутствие данных, притворяющееся данными. Их приходится вычищать всем, кто получит вашу таблицу дальше.

```
$ file-automation contacts ./company-files
Выгрузок прочитано: 4, записей: 131
Отбраковано значений: 11, записей без телефона и почты: 5
```

Задачи

1. Нормализуйте телефон: оставьте цифры и приведите начало номера к одному виду.
2. Отбракуйте значение, в котором цифр не хватает на номер.
3. Нормализуйте почту: обрежьте пробелы, приведите к нижнему регистру, проверьте `@`.
4. Нормализуйте имя: уберите двойные пробелы, приведите регистр к обычному.
5. Оставьте пустое значение пустым, а заглушки вроде `-` и `NULL` тоже сделайте пустыми.
6. Посчитайте отбракованные значения и сообщите о них.

Требования

- Телефоны, записанные с `8`, `7`, `+7`, скобками и дефисами, после нормализации совпадают.
- Нормализованный телефон это 11 цифр подряд, первая из них `7`: ни плюса, ни скобок, ни пробелов в значении не остаётся.
- Почта приведена к нижнему регистру и обрезана по краям, адрес без `@` отбракован.
- Имя не содержит двойных пробелов.
- Пустое значение остаётся пустым: ни `-`, ни `null`, ни `NULL` в результат не попадают.
- Значение, не прошедшее нормализацию, в таблицу не попадает, и о нём сообщено.

Подсказки

- Цифры отбирает `filter` по строке: `[...value].filter((symbol) => "0123456789".includes(symbol))`.
- Российский номер — это 11 цифр, начинающихся с `7`. Номер из 10 цифр приехал без кода страны.
- Почту разбирает `split("@")`: частей должно быть ровно две, и обе непустые.
- Двойные пробелы уходят через `split(" ")` и `filter`, регистр меняют `toUpperCase` и `toLowerCase`.
- Список заглушек лучше держать отдельным массивом рядом с нормализацией: он пополняется при каждой новой выгрузке.

Результат шага

`file-automation contacts ./company-files` сообщает, сколько значений отбраковано, а телефоны и адреса внутри программы приведены к одному виду.

Шаг 10

Один человек — одна строка

Контакты нормализованы, и теперь видно, что один и тот же человек приехал из нескольких источников: он был на вебинаре, лежал в старой CRM и оставил почту в рассылке. Компании нужна одна строка на человека, а не три.

Записи об одном человеке склеиваются, и поля при этом дополняют друг друга: из вебинара приехало имя, из CRM — почта, из json — компания. Выбирать между «есть имя» и «есть почта» не надо, в склеенной записи есть и то и другое.

Ключ склейки

Чтобы понять, что записи об одном человеке, нужен ключ — значение, по которому они опознаются. Ключ здесь такой: нормализованный телефон, а если телефона нет — почта.

Порядок именно такой, потому что телефон в этих выгрузках надёжнее: почту человек меняет чаще и заводит по несколько штук. А запись, в которой нет ни телефона, ни почты, опознать не по чему вовсе — она отбраковывается целиком.

Именно ради этого шага нормализация была отдельным: до неё `8 (912) 345-67-89` и `89123456789` — разные ключи, и один человек превращается в двух.

Источники

У склеенной записи перечисляются все источники, из которых она собралась. Так компания видит, откуда контакт пришёл, и понимает, какие выгрузки вообще имеют смысл.

```
$ file-automation contacts ./company-files --out ./out
Выгрузок прочитано: 4, записей: 131
Уникальных контактов: 101
Отбраковано значений: 11, записей без телефона и почты: 5
Таблица: out/contacts.csv
```

Чистая таблица — это *contacts.csv* с колонками: имя, телефон, почта, источники.

Задачи

1. Соберите ключ каждой записи: нормализованный телефон, при его отсутствии почта.
2. Отбросьте записи, у которых нет ни телефона, ни почты.
3. Склейте записи с одинаковым ключом в одну, дополняя пустые поля значениями из других записей.
4. Соберите у склеенной записи список источников.
5. Запишите *contacts.csv* в папку из `--out`.
6. Допишите в сводку число уникальных контактов и путь к таблице.

Требования

- *contacts.csv* появляется в папке из `--out` и содержит колонки: имя, телефон, почта, источники.
- Записи с одинаковым нормализованным телефоном превращаются в одну строку.
- Запись без телефона склеивается по почте.
- Запись без телефона и без почты в таблицу не попадает.
- В склеенной записи заполнены поля из всех её источников.
- Источники перечислены в отдельной колонке, каждый по одному разу, и разделены точкой с запятой с пробелом: `kontakty-vebinar.csv; kontakty-crm.csv`.
- В таблице нет двух строк с одинаковым телефоном или одинаковой почтой.

Подсказки

- `Map` с ключом контакта — это готовая склейка: если ключ уже есть, дополняете найденную запись, если нет, кладёте новую.

- Дополнить пустое поле помогает `||`: `known.почта = known.почта || record.почта`.
- Источники удобно хранить массивом и склеивать в строку только при записи в csv.
- Запись csv у вас уже написана в первой команде — используйте её же.

Результат шага

`file-automation contacts ./company-files --out ./out` создаёт *out/contacts.csv*: одна строка на человека, с перечисленными источниками. Вторая команда работает целиком.

Шаг 11

Вложенные папки

Всё это время программа смотрела только на верхний уровень папки компании. А там есть *Архив/*, и в нём лежат документы и ещё одна выгрузка контактов — их программа не видела ни разу.

Пора включить вложенность. И тут выясняется, чего стоило решение держать обход папки в одном месте.

Если обход у вас общий, работы на этом шаге — на одну функцию: вместо того чтобы пропускать папку, зайти в неё и собрать файлы оттуда тем же способом. Обе команды подхватят вложенность сами, потому что обе ходят по папке одним и тем же кодом.

Если обход разошёлся по двум командам, править придётся оба места, и это ровно тот случай, когда копипаста предъявляет счёт. Тогда сначала сведите обход в одну функцию, а потом уже делайте его рекурсивным.

Функция, которая зовёт себя

Обход вложенных папок — классическая рекурсия: функция собирает файлы папки, а встретив внутри папку, зовёт саму себя для неё и добавляет полученное к своему списку.

Глубина при этом не важна: подпапка внутри подпапки обходится тем же кодом, никаких «двух уровней» и «трёх уровней» в программе нет.

```
$ file-automation files ./company-files --out ./out
Файлов найдено: 40
Табличных документов: 21, текстовых: 8, непарсимых: 11
Документов: 28, требуют ручного разбора: 7, посторонних: 5
Копий найдено: 5 в 3 группах
Реестр: out/registry.csv
```

Число файлов выросло — из подпапки приехали и документы, и ещё одна выгрузка контактов, а значит выросло и число уникальных контактов.

Задачи

1. Найдите в коде место, где обход пропускает папки.
2. Сделайте обход рекурсивным: встретив папку, соберите файлы из неё тем же способом.
3. Убедитесь, что вложенность подхватили обе команды, а правка была одна.

Требования

- Файлы из вложенных папок попадают и в реестр, и в чистую таблицу контактов.
- Вложенность включена правкой в одном месте, а не отдельно в каждой команде.
- Глубина вложенности не ограничена: подпапка внутри подпапки тоже обходится.
- В реестре стоит путь до файла целиком, вместе с именем подпапки.

Подсказки

- `readdirSync` с `withFileTypes` уже отличает файл от папки — раньше папки просто отбрасывались.
- Собрать списки из нескольких папок в один помогает `flatMap`.
- Проверить, что правка была одна, просто: если пришлось менять две команды, обход у вас всё ещё в двух местах.

Результат шага

`file-automation files ./company-files` и `file-automation contacts ./company-files` видят файлы из *Архив/*: в реестре стало больше строк, в чистой таблице — больше контактов.